

LiquidAI / LFM2.5-Encoder-350M-Policy-Linter

like 3

Follow Liquid AI 4.42k

Token Classification
Transformers
Safetensors
15 languages
lfm2
feature-extraction

liquid
lfm2.5
bidirectional
masked-lm
encoder
custom_code
License: lfm1.0

Deploy
Copy to bucket
NEW
Use this model

Model card
Files
xet
Community 1



[Try LFM](#) • [Docs](#) • [LEAP](#) • [Discord](#)

LFM2.5-Encoder-350-Policy-Linter

A full fine-tune of [LFM2.5-Encoder-350M](#) with a rule-matching head that scores every text token against free-text policy rules in a single encoder pass.

Find more details about our encoders in our [blog post](#).

Demos: Try this fine-tuned model running in a CPU-only Hugging Face space: [Zero-shot policy linting](#)** — check text against your company's rules, written as free text. It scores every token against every rule in one pass.

Usage

Downloads last month
45



Safetensors

Model size 0.4B params

Tensor type F32
Files info

Inference Providers

Token Classification

This model isn't deployed by any Inference Provider.

 Ask for provider support

Model tree for LiquidAI/LFM2.5-E...

Base model LiquidAI/LFM2.5-350M-Ba...

Finetuned LiquidAI/LFM2.5-En...

Finetuned (17) this model

Quantizations 1 model

Space using LiquidAI/LFM2.5-E... 1

LiquidAI/policy-linting

“⚠️ Loads custom code via
`trust_remote_code=True` (the model wraps a
`trust_remote_code` encoder).”

Install the required packages:

```
pip install torch transformers
```

Run zero-shot policy linting:

```
import sys
from pathlib import Path

import torch
from transformers import AutoTokenizer

repo = Path(".")
sys.path.insert(0, str(repo))

from train_bizlint_v02 import Lfm2BidirForRuleMatching

rules = [
    "Flag direct mentions of competitor companies",
    "Flag promises about guaranteed financial performance",
]
text = "Our product is better than AcmeAI and we guarantee a 10% increase in revenue."

prefix = "Policy:\n" + "\n".join(f"- {rule}" for rule in rules)
full_text = prefix + text

tokenizer = AutoTokenizer.from_pretrained(repo / "tokenizer")
model = Lfm2BidirForRuleMatching.from_pretrained(repo / "model")

enc = tokenizer(full_text, return_offsets_mapping=True)
offsets = enc.pop("offset_mapping")[0].tolist()

rule_pool = torch.zeros(1, len(rules), len(offsets))
pos = len("Policy:\n")
```

```

for rule_idx, rule in enumerate(rules):
    start = pos + 2
    end = start + len(rule)
    token_idx = [
        i for i, (a, b) in enumerate(offsets)
        if a < end and b > start and a != b
    ]
    rule_pool[0, rule_idx, token_idx] = 1 /
    pos = end + 1

with torch.no_grad():
    probs = model(**enc, rule_pool=rule_pool)

text_start = len(prefix)

for token_idx, (a, b) in enumerate(offsets):
    if b <= text_start or a == b:
        continue

    token_text = full_text[a:b]
    for rule_idx, prob in enumerate(probs[token_idx]):
        if prob.item() > 0.5:
            print(f"{token_text!r} -> {prob!r}")

```

Contact

- Got questions or want to connect? [Join our Discord community](#).
- If you are interested in custom solutions with edge deployment, please contact [our sales team](#).

Citation

```

@article{liquidAI2026Encoders,
  author = {Liquid AI},
  title = {LFM2.5-Encoders: Fast at Long Con
  journal = {Liquid AI Blog},

```

```
year = {2026},
note = {www.liquid.ai/blog/lfm2-5-encoders}
}
```

 System theme

[TOS](#)

[Privacy](#)

[About](#)

[Careers](#)



[Models](#)

[Datasets](#)

[Spaces](#)

[Pricing](#)

[Docs](#)